have caused the cache to grow indefinitely, likely till it overflowed your system's heap.

Likewise, line 1 could bring to race conditions if two parallel calls to set the same key happen simultaneously.

It also gets worse than this. For instance, think about how we might want to fix this problem: you might be tempted to add another check right before line 7, but would that work? It would somehow lower the priority that things might go berserk, but wouldn't really solve the problem.

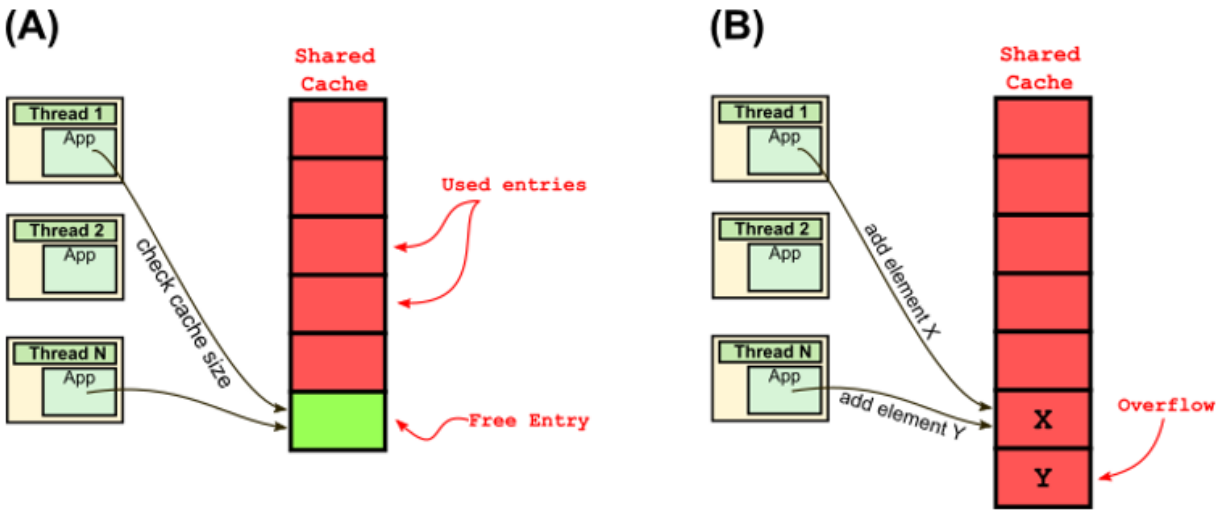


Figure 7.12 A shared cache used in a multithreaded environment, without implementing synchronization. (A) Two thread concurrently try to add a new element to a cache with only one free spot. When the `LFUCache::set` method inside each thread concurrently checks for cache size (line #5 in listing 7.8), they both see it's not full.
(B) Without synchronization, both threads add an element to the cache (line #7), without evicting; depending on the implementation of the cache (and of